DrainSinkhorn: Dynamic Compaction for Batched Entropic Optimal Transport*

Xinyang Wen

Abstract

Fast Sinkhorn kernels reduce the cost of each optimal transport update, but a fixed-width batch continues to execute completed problems until its slowest member finishes. We present DrainSinkhorn, an execution layer that removes completed problems and narrows subsequent updates. A one-marginal screen selects candidates for a two-marginal verifier; verified outputs are emitted, and the remaining per-problem state is compacted. A hardware-aware performance model relates completion iterations to active batch width, measured kernel time, and control overhead. The model explains why reducing problem updates produces different runtime gains across problem sizes. On MetroPT-3, DrainSinkhorn reduces problem updates from 5696 to 3128 and achieves 1.746× pipeline speedup with four GPU workers. On ImageNet-32, it achieves 1.054× Sinkhorn speedup and 1.036× over full training. Packer19 component tests isolate the effects of screening, verification, and physical compaction, while OTT-JAX and PyKeOps implementations demonstrate removal of completed problems across backends.

1 Introduction

Entropic optimal transport (EOT) supplies couplings for industrial sensor analysis, single-cell trajectory inference, and flow-matching training. GPU implementations accelerate Sinkhorn updates through matrix-free operators, tiled reductions, and fused kernels [4, 5, 18]. These applications often require multiple independent couplings, which can share a batched kernel.

The problems in a batch reach the stopping tolerance at different iterations. A fixed-width kernel nevertheless retains every problem until the slowest one finishes. Completed problems continue to occupy kernel lanes and verification bandwidth. Logical masking freezes their state while preserving the original tensor width. As inner kernels become faster, the execution of these completed problems becomes a separate source of cost.

Our key observation is that each verified completion provides an opportunity to shrink the next update. Sinkhorn problems are independent along the batch dimension, so removing one problem preserves the update applied to the remaining problems. Real workloads contain enough variation in completion iterations for this removal to matter: in the Packer19 component test, physical compaction reduces executed problem updates from 256 to 156 (figure 1 and table 5).

We introduce DrainSinkhorn, an execution layer that performs screen, verify, and compact during batched Sinkhorn. After a complete scaling cycle, one marginal is current in exact arithmetic. A batched screen evaluates the other marginal and selects candidates for the full two-marginal verifier. The executor emits verified outputs and compacts potentials, marginals, support descriptors, and scratch buffers together. Subsequent kernels process the unfinished batch.

The runtime benefit depends on the GPU cost of the resulting widths. A smaller batch can launch fewer blocks while still occupying the same number of scheduling waves. We therefore model runtime using completion iterations, measured update time at each active width, and control overhead. The model connects the amount of removable work to the hardware conditions under which compaction saves time.

We evaluate this design on MetroPT-3, ImageNet-32, Packer19, and A2D2. The primary MetroPT-3 experiment achieves 1.746× pipeline speedup. ImageNet-32 achieves 1.054× Sinkhorn speedup and 1.036× over full training. Component tests identify the effects of screening, verification, and compaction; width and grouping experiments explain variation in runtime benefit. OTT-JAX and PyKeOps implementations test the execution principle across backends.

This paper makes three contributions. First, DrainSinkhorn combines Sinkhorn-specific screening with verified completion and physical compaction of batched state. Second, a hardware-aware performance model relates removed updates to kernel time and control overhead. Third, application and component experiments establish the runtime effects, output quality, and operating conditions of the execution layer.

*Code: github.com/cis-aconiticacid/DrainSinkhorn

2 Background and Motivation

2.1 Batched entropic optimal transport

For positive unit-mass marginals $a \in \mathbb{R}^n_+$ and $b \in \mathbb{R}^m_+$, EOT solves

$$\min_{\pi \geq 0} \langle C, \pi \rangle + \varepsilon \sum_{ij} \pi_{ij}(\log \pi_{ij} - 1), \quad \pi \mathbf{1} = a, \quad \pi^\top \mathbf{1} = b. \tag{1}$$

Here C is the transport cost, $\varepsilon > 0$ is the regularization parameter, and π is the coupling. With $K = \exp(-C/\varepsilon)$, Sinkhorn alternates diagonal scalings to form $\pi = \mathrm{diag}(u) K \, \mathrm{diag}(v)$ [3]. A batched solver applies these updates to several independent problems.

Coordinate, relaxed, low-rank, and Newton methods change the work inside an individual solve [1, 8, 14, 15]. Matrix-free and fused implementations reduce memory traffic, including the stabilized LogSumExp updates of FlashSinkhorn [18]. DrainSinkhorn operates around the selected update kernel and changes the set of problems receiving its next invocation.

2.2 Completed problems consume batch work

Fixed-width execution presents the original batch to every update. When some problems have passed verification, later kernels still contain their lanes. The Packer19 component experiment makes this cost visible: logical masking retains 256 problem-update slots, whereas dynamic compaction executes 156 (figure 1). This comparison uses the same observed completion decisions; section 5.4 gives the measurement settings.

Fewer update slots need not produce a proportional reduction in GPU time. The kernel's grid depends on batch width and problem size, and GPU scheduling groups blocks into waves. A width reduction saves kernel time when it shortens this execution. This motivates both an executor that removes completed problems and a model that accounts for the measured cost at each remaining width.

2.3 Related execution systems

Dynamic batching already supports data-dependent execution in other settings. Automatic batching groups program members by execution point [13], and Orca schedules autoregressive inference one iteration at a time [20]. Ginkgo's batched conjugate-gradient solver skips converged systems inside its fused kernel [7]; jax-ipm combines heterogeneous iterations and replacement of completed problems [17]. Within OT, POT provides `solve_batch`, and OTT-JAX supports vectorized Sinkhorn solves [4, 6, 12].

DrainSinkhorn applies completion-driven compaction to matrix-free EOT batches. Its packed Triton implementation uses the alternating-scaling structure for screening

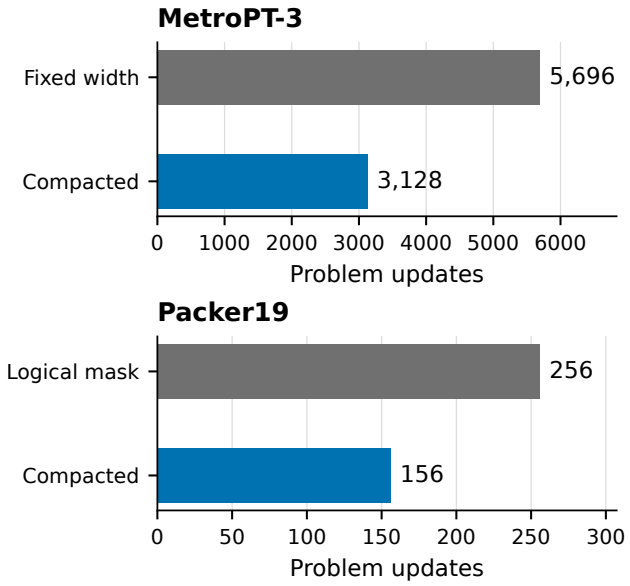

Figure 1: Problem updates before and after compaction. Each panel uses its own workload: the primary four-worker MetroPT-3 study and the Packer19 logical-mask control. Counts are totals for the respective measured execution.

and preserves the correspondence among all per-problem tensors during compaction. OTT-JAX and PyKeOps implementations use their own verification and packing mechanisms.

Initialization also affects batch execution by changing completion iterations. Carried potentials, analytic extensions, and learned proposals reduce the initial residual [2, 16]; soft c-transforms support changing couplings in flow matching [21]. Our experiments include cold-start couplings and a controlled setting with shared initialization. Downstream tasks consume the resulting coupling, for example to select flow-matching training pairs [11]. Their total runtime includes both the solver and this consumer work.

3 DrainSinkhorn

3.1 Problem and execution state

Consider a batch of W independent EOT problems from equation (1). A lane contains one problem's marginals, support descriptors, dual potentials, and temporary state. An active problem is one that has yet to pass the configured stopping rule. DrainSinkhorn maintains a packed array of active problems and their output destinations.

The executor advances this array until every problem has completed or the configured iteration limit is reached. Its central invariant is that all tensors at an active lane refer to the same problem. Each emitted output must also pass the two-marginal verifier. These two conditions govern screening, output release, and compaction.

3.2 Screen, verify, and compact

Each iteration applies a batched Sinkhorn cycle to the active problems. At the verification interval, the executor screens every active lane, verifies screen-positive candidates, emits passing outputs, and compacts the unfinished state. The verification interval counts the Sinkhorn cycles between these checks.

For one problem, a Sinkhorn cycle computes

$$u^{k+1} = a \oslash (Kv^k), \qquad v^{k+1} = b \oslash (K^\top u^{k+1}), \quad (2)$$

where $\oslash$ denotes elementwise division. Immediately after the second scaling, exact arithmetic gives

$$v^{k+1} \odot (K^\top u^{k+1}) = b. \quad (3)$$

The updated column marginal therefore satisfies its target, while the row marginal $u^{k+1} \odot (Kv^{k+1})$ may still have a residual. The screen batches this row-residual calculation and selects a problem when $\widehat{r}^{\mathrm{row}}_{t,k} \le \tau_{\mathrm{screen}}$, where τ_{screen} is the screening threshold.

The full verifier recomputes both marginals of each selected coupling $\pi_{t,k}$ and tests

$$r_{t,k} = \max\left\{\|\pi_{t,k}\mathbf{1} - a_t\|_1,\ \|\pi_{t,k}^\top \mathbf{1} - b_t\|_1\right\} \le \tau_{\mathrm{stop}}, \quad (4)$$

where τ_{stop} is the stopping tolerance. Passing problems are emitted; rejected candidates remain active. A screen false positive adds a verifier call, and a false negative postpones compaction. The component experiment measures these effects by replaying screening decisions with the full verifier.

3.3 Compacting independent state

After verification, the executor applies the same selection to support descriptors, marginals, potentials, centered shifts, log weights, and scratch buffers. It preserves each problem's output destination as lanes move. This aligned selection produces the narrower inputs for the next backend call.

Let F_t be one Sinkhorn cycle on the complete state z_t of problem t. Because batching adds a problem dimension with independent updates, ideal arithmetic gives

$$\widetilde{F}(\mathrm{pack}(z_1, \ldots, z_W)) = \mathrm{pack}(F_1(z_1), \ldots, F_W(z_W)). \quad (5)$$

Selecting a subset of the packed problems therefore preserves the update of each remaining problem. The output verifier enforces the stopping rule after finite-precision execution; section 5.6 reports residuals and output checks.

3.4 Backend execution

The packed Triton implementation applies screening and verification at the configured interval and compacts all matrix-free solver state. A tail threshold controls handoff to its single-problem solver; MetroPT-3 and Packer19 set this threshold to one. Iteration limits and non-finite states terminate with failure. Runtime includes the configured control and handoff operations.

The OTT-JAX implementation verifies at fixed phase boundaries, removes completed problems, and rebuckets the remaining batch. PyKeOps uses a batched independent-problem operator with a geometric verification schedule and removal of completed state. When a problem fits on one GPU, independent workers process different windows. The scale-out experiments use one worker per GPU.

4 Hardware-Aware Performance Model

The model connects completion iterations to executed work and runtime. Its inputs are the observed completion iteration of each problem, the measured update cost at each batch width, and the control overhead. This use of workload shape and device cost follows accelerator performance modeling [9, 19]. The model organizes the width, component, and grouping experiments in section 5.

4.1 Completion iterations determine active width

Let n_t be the iteration of the first configured verification at which problem t passes, and let $n_{\max} = \max_t n_t$. Before update k, the active width is

$$A_k = \left|\{t : n_t \ge k\}\right|. \quad (6)$$

These iteration counts include the effect of the verification interval. A more frequent schedule can observe completion earlier and produce a different trajectory.

For the same completion decisions, fixed-width execution uses $Wn_{\max}$ problem-update slots. Compaction uses

$$S_{\mathrm{active}} = \sum_{k=1}^{n_{\max}} A_k = \sum_{t=1}^{W} n_t, \qquad S_{\mathrm{static}} = Wn_{\max}. \quad (7)$$

The identity follows by counting each problem once at every iteration before its completion. The removed work is

$$R = Wn_{\max} - \sum_t n_t. \quad (8)$$

Geometrically, R is the area between the fixed-width rectangle and the active-width trajectory. In the matched logical-mask control, completed states are frozen while the launched width remains W.

A batch has removable work when completed and unfinished problems coexist before an update. Thus variation within a batch matters: identical completion iterations yield $R = 0$, even if completion iterations differ across batches. The grouping experiment changes this within-batch variation using a fixed set of problems.

4.2 Kernel savings must cover control cost

Let $c_k(a, \xi_k)$ denote the measured cost of update k at width a. The state ξ_k specifies support shape, data type, kernel configuration, and other conditions of the comparison. The compacted runtime is

$$T_{\text{active}} = \sum_k c_k(A_k, \xi_k) + H_{\text{screen}} + H_{\text{verification}} + H_{\text{compact}} + H_{\text{other}}. \tag{9}$$

The H terms account for screening, verification, compaction, and remaining timed control work. Write their sum as H_{active}, and let $\Delta H = H_{\text{active}} - H_{\text{fixed}}$ be the incremental control cost relative to fixed-width execution.

For a comparison with the same completion decisions and matched update states, subtracting the two runtime accounts gives

$$T_{\text{fixed}} - T_{\text{active}} = \sum_{k=1}^{n_{\max}} [c_k(W, \xi_k) - c_k(A_k, \xi_k)] - \Delta H. \tag{10}$$

Compaction reduces runtime when

$$\sum_k [c_k(W, \xi_k) - c_k(A_k, \xi_k)] > \Delta H. \tag{11}$$

The width sweep measures the update-cost term, and component measurements quantify the additional control work. This equation explains the break-even condition for the measured execution path.

4.3 GPU scheduling makes width cost stepwise

For the packed Triton update, width a and support size n produce

$$B(a, n) = a \left\lceil \frac{n}{b_m} \right\rceil \tag{12}$$

thread blocks, where b_m is the row tile size. GPU scheduling executes blocks in waves whose capacity depends on occupancy [9, 10]. A model of this cost is

$$c(a; n, \xi) \approx g_{n,\xi}\left(\left\lceil \frac{B(a, n)}{Q_\xi} \right\rceil \right), \tag{13}$$

where Q_ξ is the resident block capacity for the device and kernel, and $g_{n,\xi}$ maps wave count to elapsed time. Registers and shared memory affect Q_ξ.

Reducing width within a scheduling wave can leave update time nearly unchanged. Larger problems launch more blocks per lane, so removing a lane can reduce the number of waves earlier. Section 5.3 measures this relation on A100 and uses it to interpret the application results.

5 Experiments

The evaluation asks how much DrainSinkhorn reduces application runtime, which execution components produce the gain, and how completion variation and GPU scheduling determine its operating regime. We first define the tasks and comparisons, then report application results, kernel measurements, component tests, grouping effects, and output checks.

5.1 Experimental setup

We evaluate four tasks: MetroPT-3 for industrial sensor streams, ImageNet-32 for flow-matching training, Packer19 for single-cell trajectory inference, and A2D2 for LiDAR point-cloud matching. Table 1 gives the primary configurations. The packed Triton studies use NVIDIA A100-SXM4-80GB GPUs and verify every four Sinkhorn cycles. ImageNet-32 training uses PCA500 feature couplings, three paired seeds, and 50,000 training updates; generation is evaluated at 4, 8, and 16 function evaluations (NFE).

We measure E_1 (Sinkhorn time) through the verified coupling or dual potentials, E_2 (pipeline time) through the downstream consumer, and E_3 (total time) through the full training, generation, or task-evaluation interval. Speedup is baseline time divided by DrainSinkhorn time. In the two-host MetroPT-3 and Packer19 component tests, E_1 is obtained by subtracting paired consumer time within each observation before computing ratios and uncertainty.

Fixed-width execution keeps the original batch through the final verification. Logical masking (LM) freezes completed states at the original width. Dynamic compaction (DC) removes completed state and narrows the batch. Fixed/DC measures the complete execution change; LM/DC isolates compaction under the same observed release decisions. Component tests fix inputs, initialization, tolerance, verification schedule, verifier, backend, and consumer. Repeats characterize timing variability; training seeds, clips, and host/order/GPU combinations provide the statistical units specified with each result.

5.2 Application runtime and backend transfer

DrainSinkhorn reduces both solver time and application time, with the size of the benefit depending on the workload. Table 2 separates the three timing levels. The primary MetroPT-3 pipeline achieves 1.746× speedup with four independent GPU workers as problem updates fall from 5696 to 3128. All three execution orders favor compaction, with an observed range of 1.7453–1.7460×. The corresponding ratio of fixed to compacted GPU energy is 1.779×.

ImageNet-32 gains are smaller. Across the same three paired training seeds, the Sinkhorn-time ratio is 1.054× and the full-training ratio is 1.036×. Table 4 reports

Table 1: Primary task settings. The packed Triton paths use cold initialization. Component and stopping studies use their own observation units, as specified in the text.

Task	Backend	Endpoint	n	W	Numerical settings
MetroPT-3	Triton	Pipeline	16384	16	$\varepsilon = 0.1,\ \tau = 10^{-3}$
ImageNet-32	Triton	Training	1024	8	$\varepsilon = 0.03,\ \tau = 10^{-3}$
Packer19	Triton	Transition	16384	8	$\varepsilon = 0.1,\ \tau = 10^{-4}$–$10^{-2}$
A2D2	PyKeOps	Matching	8192	8	Three LiDAR clips

Table 2: Application results grouped by timing level. Ratios are baseline/DC. Ranges in parentheses describe observed seeds or execution orders; brackets denote bootstrap 95% intervals. Each row retains its own observations and is not pooled with other rows.

Scope	Task	Baseline	Speedup	Observations
E_1	ImageNet-32	Fixed	1.054 (1.033–1.072)	3 training seeds
E_1	Packer19 components	LM	1.540 [1.5389, 1.5404]	24 paired units
E_2	MetroPT-3	Fixed	1.7455 (1.7453–1.7460)	3 orders; 4 workers
E_3	ImageNet-32	Fixed	1.036 (1.030–1.043)	3 training seeds

feature-space quality after training, and table 7 gives absolute solver times by seed. A separate coupling-preparation diagnostic gives 1.105×. With one initialization shared across 40 controlled overlapping-support windows, compaction gives 1.421× and removes 30.62% of updates.

Removal of completed problems also reduces time in independent backend implementations (table 3). OTT-JAX gives 2.600× relative to native vectorized execution. In the same backend study, its matched fixed-phase comparison gives 1.545×. A separate tolerance sweep gives 1.366–1.581× at tolerances from 10^{-3} to 5×10^{-3} and 0.995× at 10^{-2}. The PyKeOps comparisons against sequential execution give 3.174× on A2D2 and 1.415× on ImageNet-32. These contrasts evaluate the implementations named in the table.

The MetroPT-3 scale-out study assigns complete windows to eight workers on two hosts. Fixed-work-per-worker pipeline ratios are 3.110× and 2.159× for its two inputs. A separate one-to-four-worker study achieves 3.965–3.979× throughput scaling on held-out routes. Table 9 lists the absolute eight-worker endpoint times and support-size results.

5.3 Kernel time by active batch width

The width experiment tests the update-cost term in equation (10). It varies active width while holding the packed Triton update and problem size fixed. Each point in figure 2 summarizes 30 direct primitive calls; the timing covers the two update kernels. Screening, verification, and compaction are measured in the component study.

At $n = 1024$, widths one through six have similar update time, followed by a large increase from six to seven. At $n = 4096$, the first large increase occurs from one to two; the $n = 16384$ curve responds from width one. With row tile size $b_m = 64$ and 108 SMs, the one-block-per-SM wave-width estimate $b_m N_{\mathrm{SM}}/n$ is 6.75, 1.69, and 0.42 for these three sizes. The observed steps are consistent with the scheduling-wave explanation.

The small-problem plateau helps interpret the modest ImageNet-32 solver gain: lane removal within the plateau reduces update slots with little change in primitive time. Larger problems have a stronger width response, consistent with the larger MetroPT-3 gains. The application endpoint additionally includes control and consumer work, as expressed in equation (10).

5.4 Packer19 single-variable component tests

The Packer19 study asks which parts of the execution layer reduce time. It uses one transition workload on two hosts with four A100 GPUs each. Three method orders produce 24 paired observations indexed by host, order, and GPU. Timings are medians after warm-up. As shown in Table 5, each contrast changes one component at tolerance 10^{-3}.

Batched verification and the one-marginal screen reduce OT-stage time by factors of 1.133 and 1.105, respectively. All 24 paired observations favor each change. Replaying 1488 screen-negative events with the full verifier finds 0 already-completed problems; the maximum row-residual discrepancy is 1.86×10^{-7}.

Logical masking leaves the packed width at eight and yields a ratio near one. Compaction reduces problem updates from 256 to 156 and gives 1.540× speedup, with interval 1.5389–1.5404×. Across paired-observation medians, GPU energy falls from 879.9 to 688.1 J, while peak allocated memory increases from 80.8 to 106.9 MiB. Table 8 gives the separately summarized phase times.

A separate stopping-tolerance sweep tests the same LM/DC comparison over five tolerances. Figure 3 shows

Table 3: Backend implementations and their comparison paths. All rows measure Sinkhorn time. Parentheses denote the observed repeat, clip, or seed range indicated in the last column. Residuals are maximum marginal L_1 errors.

Task / backend	Baseline	Speedup	Residual	Unit
ImageNet-32 / OTT-JAX	Native vectorized	2.600 (2.338–2.664)	$9.97\,10^{-4}$	6 repeats
A2D2 / PyKeOps	Sequential carry	3.174 (2.180–3.944)	$9.97\,10^{-4}$	3 clips
ImageNet-32 / PyKeOps	Sequential	1.415 (1.360–1.448)	$8.93\,10^{-4}$	3 seeds

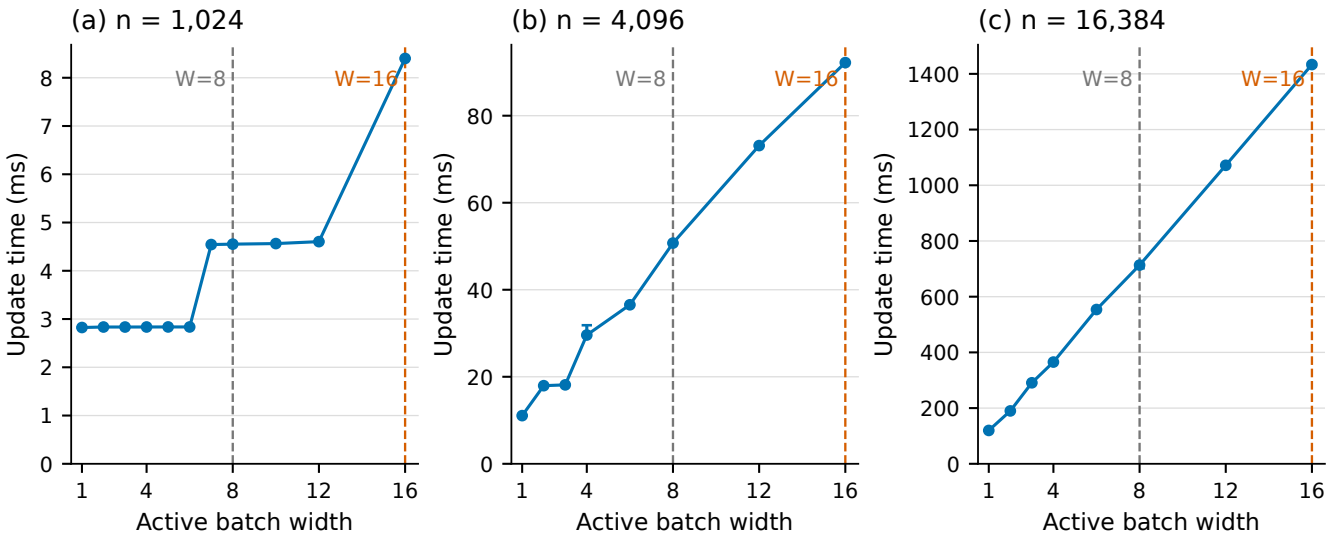

Figure 2: A100 packed-update time versus active batch width. Points show medians and whiskers show interquartile ranges from 30 repeats. Each panel uses its own vertical scale in milliseconds. Vertical guides locate initial widths $W = 8$ and $W = 16$. The $n = 1024, W = 8$ configuration matches the ImageNet-32 training solver with $\varepsilon = 0.03$, verification interval 4, and tolerance 10^{-3}.

Table 4: ImageNet-32 quality after 50,000 updates in PCA500 feature space. Entries are three-seed means; lower values are better. Held-out flow-matching MSE has the same reported value at all three NFE settings.

Metric	Execution	NFE 4	NFE 8	NFE 16
Curvature	Fixed	5.4524	2.8388	1.4546
	Compacted	5.4448	2.8356	1.4528
Projected Fréchet	Fixed	1.2967	1.5286	1.8182
	Compacted	1.2886	1.5224	1.8136
Held-out FM MSE	Fixed		1.07854	
	Compacted		1.07859	

speedup alongside achieved residual and consumer total-variation distance (TV) from the tight reference. Compaction gives 1.480–1.638× from 10^{-4} through 3×10^{-3}. At 10^{-2}, all eight problems first pass at update 8; the ratio is 0.9995× with 95% interval [0.9987, 1.0003]. This equal-completion case exercises the $R = 0$ condition in the model.

Table 5: Single-variable Packer19 component comparisons at $\tau = 10^{-3}$. Ratios use OT-stage time after paired consumer subtraction. All four comparisons use 24 paired observations. Intervals shown are bootstrap 95% intervals; the first two contrasts report paired wins.

Change	Ratio	Evidence
Batched verification	1.133	24/24 faster
One-marginal screen	1.105	24/24 faster
Logical mask	1.000	[0.9998, 1.0003]
Physical compaction	1.540	[1.5389, 1.5404]

5.5 MetroPT-3 grouping test

The grouping test asks whether completion variation within a batch changes compaction benefit. It holds 32 real MetroPT-3 problems fixed and changes their assignment to windows. Offline every-cycle completion iterations q_t define the grouping; verifier-derived iterations n_t measure the executed work. Homogeneous grouping places problems with similar completion iterations together, while heterogeneous and random layouts create more completed lanes inside unfinished batches.

As shown in Table 6, the fixed/DC ratio is 1.105× for ho-

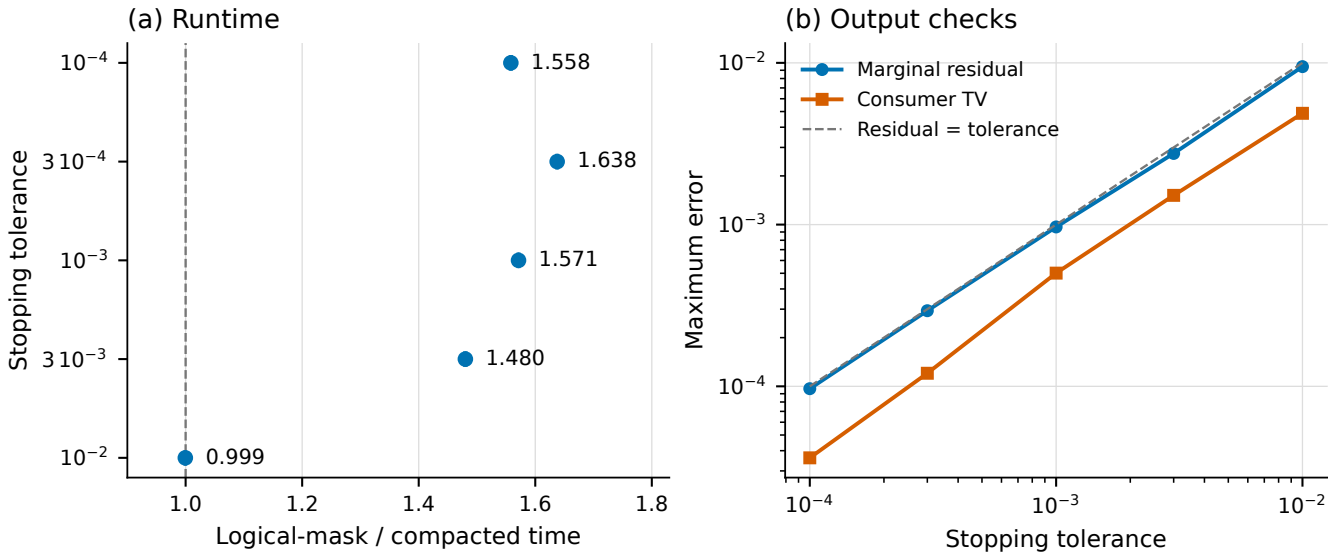

Figure 3: Packer19 compaction across stopping tolerances. Left: LM/DC speedup with equal-host stratified bootstrap 95% intervals over 24 paired observations per tolerance; the vertical line marks equal runtime. Right: maximum achieved marginal residual and maximum consumer TV against the tight reference. The dashed diagonal marks residual equal to the requested tolerance.

Table 6: Effect of grouping 32 fixed MetroPT-3 problems. The update fraction is dynamic/fixed problem updates; speedup is fixed/dynamic time. Both quantities summarize execution of the same problem set under each assignment.

Grouping	Update fraction	Speedup
Homogeneous	0.892	1.105
Heterogeneous	0.802	1.217
Random 1	0.776	1.258
Random 2	0.789	1.235
Random 3	0.764	1.271

mogeneous grouping, 1.217× for heterogeneous grouping, and 1.235–1.271× for random layouts. The corresponding dynamic-update fractions are 0.892, 0.802, and 0.764–0.789. These comparisons show how batch assignment changes removable work for the same problem set.

5.6 Numerical accuracy and application outputs

The timed packed Triton applications use the backend precision and the two-marginal verifier. All emitted plans meet the recorded marginal-residual bounds, and all reported consumer and training-quality checks pass. On primary MetroPT-3, maximum row and column residuals are 9.476650×10^{-4} and 1.283915×10^{-6}; both execution modes attain failure-window ROC AUC 1.0. ImageNet-32 quality is evaluated in PCA500 feature space at NFE 4, 8, and 16 (table 4).

An independent finite-precision stress test covers 84 near-threshold problems and 16 runtime stress cases over 1/2/4/8 GPUs. Against FP64 replay, raw FP32 and TF32 each disagree on three acceptance decisions. An outward-bound guard followed by FP64 escalation has zero disagreements in these 100 cases. This guarded configuration is evaluated separately from the timed applications.

6 Discussion

DrainSinkhorn benefits batches whose members complete at different verification rounds and whose narrower kernels save enough time to cover screening, verification, and state movement. The grouping test and width sweep examine these two conditions separately. Initialization and batch assignment affect completion iterations, while problem size and kernel configuration determine the time associated with each active width.

Compaction adds buffers and data movement. The Packer19 component test measures 26.1 MiB of additional peak allocation together with lower runtime and GPU energy; table 8 reports its phase costs. Backend implementations can use different packing and tail-handoff strategies, so their full execution costs include these choices.

The performance model uses a width-cost function for a specified device, kernel, problem shape, and precision. The reported scheduling-wave behavior describes the measured A100 Triton configuration. Applying the model to another backend requires its corresponding width-cost measurements. Offline completion counts also support analysis of batch grouping; using estimated counts for online grouping remains a separate scheduling problem.

7 Conclusion

DrainSinkhorn removes completed problems from batched Sinkhorn execution through screening, verification, and physical compaction. The resulting active-width trajectory reduces problem-update slots, while measured kernel cost and control overhead determine runtime benefit. Application experiments report solver, pipeline, and training time; component and grouping tests explain when compaction helps. Together, these results establish completion-driven batch execution as a way to reduce the cost of repeated EOT solves across several backends.

References

[1] Jason Altschuler, Jonathan Niles-Weed, and Philippe Rigollet. Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://papers.nips.cc/paper_files/paper/2017/hash/491442df5f88c6aa018e86dac21d3606-Abstract.html.

[2] Brandon Amos, Giulia Luise, Samuel Cohen, and Ievgen Redko. Meta optimal transport. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 791–813, 2023. URL https://proceedings.mlr.press/v202/amos23a.html.

[3] Marco Cuturi. Sinkhorn distances: Lightspeed computation of optimal transportation distances. In *Advances in Neural Information Processing Systems*, volume 26, pages 2292–2300, 2013. URL https://arxiv.org/abs/1306.0895.

[4] Marco Cuturi, Laetitia Meng-Papaxanthos, Yingtao Tian, Charlotte Bunne, Geoff Davis, and Olivier Teboul. Optimal transport tools (OTT): A JAX toolbox for all things wasserstein, 2022. URL https://arxiv.org/abs/2201.12324.

[5] Jean Feydy, Thibault Séjourné, François-Xavier Vialard, Shun ichi Amari, Alain Trouvé, and Gabriel Peyré. Interpolating between optimal transport and MMD using sinkhorn divergences. In *Proceedings of the 22nd International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 2681–2690, 2019. URL https://proceedings.mlr.press/v89/feydy19a.html.

[6] Rémi Flamary, Nicolas Courty, Alexandre Gramfort, Mokhtar Z. Alaya, Aurélie Boisbunon, Stanislas Chambon, Laetitia Chapel, Adrien Corenflos, Kilian Fatras, Nemo Fournier, Léo Gautheron, Nathalie T. H. Gayraud, Hicham Janati, Alain Rakotomamonjy, Ievgen Redko, Antoine Rolet, Antony Schutz, Vivien Seguy, Danica J. Sutherland, Romain Tavenard, Alexander Tong, and Titouan Vayer. POT: Python optimal transport. *Journal of Machine Learning Research*, 22(78):1–8, 2021. URL https://www.jmlr.org/papers/v22/20-451.html.

[7] Ginkgo Project. Batched CG. Ginkgo user guide, 2026. URL https://gko-project.org/user-guide/concepts/batched/cg.html. Accessed 2026-09-08.

[8] Mete Kemertas, Amir massoud Farahmand, and Allan D. Jepson. A truncated newton method for optimal transport, 2025. URL https://arxiv.org/abs/2504.02067.

[9] *GPU Performance Background User's Guide*. NVIDIA, 2022. URL https://docs.nvidia.com/deeplearning/performance/dl-performance-gpu-background/index.html.

[10] *CUDA C++ Programming Guide*. NVIDIA, 2024. URL https://docs.nvidia.com/cuda/archive/12.5.0/cuda-c-programming-guide/index.html.

[11] Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, Brandon Amos, Yaron Lipman, and Ricky T. Q. Chen. Multisample flow matching: Straightening flows with minibatch couplings. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28100–28127, 2023. URL https://proceedings.mlr.press/v202/pooladian23a.html.

[12] POT developers. POT 0.9.6 release notes. Python Optimal Transport documentation, 2025. URL https://pythonot.github.io/releases.html.

[13] Alexey Radul, Brian Patton, Dougal Maclaurin, Matthew D. Hoffman, and Rif A. Saurous. Automatically batching control-intensive programs for modern accelerators. In *Proceedings of Machine Learning and Systems*, volume 2, 2020. URL https://proceedings.mlsys.org/paper_files/paper/2020/hash/39945d578f616735572174bf5e8f155d-Abstract.html.

[14] Meyer Scetbon, Marco Cuturi, and Gabriel Peyré. Low-rank Sinkhorn factorization. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9344–9354, 2021. URL https://proceedings.mlr.press/v139/scetbon21a.html.

[15] Alexis Thibault, Lénaïc Chizat, Charles Dossal, and Nicolas Papadakis. Overrelaxed Sinkhorn–Knopp algorithm for regularized optimal transport, 2017. URL https://arxiv.org/abs/1711.01851.

[16] James Thornton and Marco Cuturi. Rethinking initialization of the sinkhorn algorithm. In *Proceedings of the 26th International Conference on Artificial Intelligence and Statistics*, volume 206 of *Proceedings of Machine Learning Research*, pages 8682–8698, 2023. URL https://proceedings.mlr.press/v206/thornton23a.html.

[17] John Viljoen, Johanna Haffner, Masayoshi Tomizuka, and Negar Mehr. Scaling nonlinear optimization: Many problems one GPU, 2026. URL https://arxiv.org/abs/2606.26341.

[18] Felix X.-F. Ye, Xingjie Li, An Yu, Ming-Ching Chang, Linsong Chu, and Davis Wertheimer. FlashSinkhorn: IO-aware entropic optimal transport on GPU, 2026. URL https://arxiv.org/abs/2602.03067.

[19] Geoffrey X. Yu, Yubo Gao, Pavel Golikov, and Gennady Pekhimenko. Habitat: A runtime-based computational performance predictor for deep neural network training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 503–521, 2021. URL https://www.usenix.org/conference/atc21/presentation/yu.

[20] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation*, pages 521–538, 2022. URL https://www.usenix.org/conference/osdi22/presentation/yu.

[21] Stephen Zhang, Alireza Mousavi-Hosseini, Michal Klein, and Marco Cuturi. On fitting flow models with large sinkhorn couplings, 2025. URL https://arxiv.org/abs/2506.05526.

A Measurement Details

A.1 Timing and statistical units

The primary MetroPT-3 endpoint is the maximum pipeline time across four independent GPU workers. Each of three execution orders is summarized by the median of two timing repeats. The reported 1.745535× is the descriptive geometric mean of the three order ratios. Both paths use the same input, initialization, regularization, tolerance, verification schedule, backend, and consumer. Each repeat invokes and evaluates 16 verifier candidates and launches 32 plan-application kernels.

The two-host MetroPT-3 comparison uses three order-balanced plans on each host, with four GPU placements per host. Its six host/plan combinations are paired timing units for a fixed workload; placements and repeats stabilize timing. The E_2 ratio is 1.744723×, with stratified bootstrap interval $[1.743410, 1.746706]$. Subtracting each unit's paired consumer time gives the E_1 ratio 1.754×. The primary four-worker and two-host comparisons use their own endpoints and observations.

GPU energy is the change in the A100 NVML cumulative energy counter between the start of the method call and the final CUDA synchronization. The counter reports millijoules, converted to joules. This interval starts with prepared arrays and includes solver, initialization, and consumer work; compilation, warm-up, and prepared-input transfer occur before it. Unsupported counters are recorded as unavailable.

A.2 Training and output measurements

ImageNet-32 uses paired seeds 20260891, 20260892, and 20260893 throughout training and NFE evaluation. Table 7 reports each seed's solver time. The full-training ratio uses the complete 50,000-update interval. The separate coupling-preparation diagnostic gives paired ratios 1.156, 1.082, and 1.079, with fixed/DC update counts 64/52, 64/56, and 64/56.

Table 7: ImageNet-32 solver times by training seed. The geometric mean of paired fixed/compacted ratios is 1.054.

Measurement	Seed 20260891	Seed 20260892	Seed 20260893
Fixed time (s)	123.429	124.233	118.840
Compacted time (s)	116.699	115.934	115.036
Fixed / compacted	1.058	1.072	1.033

All 48 worker-level primary MetroPT-3 repeats converge. The solver receives sensor inputs without failure labels. Maximum paired differences in transport cost and barycentric shift are 0.02792263 and 0.00346756 under the fixed failure-window consumer.

The Packer19 tolerance study uses two hosts, four GPUs per host, three method orders, and three timing repeats. Each tolerance has 24 paired observations indexed by host, order, and GPU, with an equal-host bootstrap interval. Consumer TV compares outputs to the 10^{-6} reference. Absolute OT-stage medians at 10^{-3} are 2.038 s for logical masking and 1.299 s for compaction in this stopping study. The component study's phase medians in table 8 are summarized independently and belong to its own LM/DC comparison.

Table 8: Packer19 component-study phase times in milliseconds. Entries are separately computed phase medians. Whole-path ratios in Table 5 come from paired timings.

Execution	Correction	Batched verification	Verifier	Mask / compact
Logical mask	1706.3	638.5	104.5	0.306
Compacted	1070.8	400.6	104.5	3.370

A.3 Width measurements and backend settings

The width sweep uses an A100-SXM4-80GB with 108 SMs, driver 535.129.03, PyTorch 2.5.1 with CUDA 12.4, and Triton 3.1.0. Each primitive call contains two packed LogSumExp update kernels. Widths are randomized within 30-repeat blocks after warm-up. The $n = 1024$ sweep covers widths 1–8, 10, 12, and 16; the other sizes cover 1, 2, 3, 4, 6, 8, 12, and 16.

OTT-JAX verifies at phase boundaries and rebuckets unfinished problems. The tolerance sweep uses one fixed ImageNet-32 input, ten timing repeats per tolerance, and phase sizes selected on disjoint warm-up inputs: 20 updates at 10^{-3} and 10 updates at the other reported tolerances. PyKeOps uses a matrix-free batched operator, geometric verification schedule, and removal of completed state. Its tiled-log implementation reaches support size $n = 65536$.

The shared-initialization experiment uses a constructed stream with 90% support overlap over real ImageNet features. Its warm replay includes descriptor, transfer, initialization, correction, and verification costs; compilation, cache loading, and process startup occur before measurement. The reported result covers 40 controlled windows.

A.4 Worker scaling and grouping

Independent GPU workers process different windows without a Sinkhorn collective. The one-to-four-worker throughput experiment holds work per worker fixed. The eight-worker experiment compares fixed-width and compacted execution at that worker count. Table 9 gives the two eight-worker inputs and the separate single-worker support-size comparisons. The reported support-size configurations meet the prespecified consumer criteria on both hosts.

Table 9: Additional MetroPT-3 runtime measurements. Eight-worker rows use fixed work per worker and pipeline time; support-size rows are separate single-worker comparisons summarized by equal-host geometric means. Energy ratios use the corresponding eight-worker endpoint.

Configuration	Fixed (s)	Compacted (s)	Time ratio	Energy ratio
8 workers, higher workload	12.523	4.026	3.110	3.437
8 workers, lower workload	7.999	3.705	2.159	2.292
1 worker, $n = 8192$	—	—	2.316	—
1 worker, $n = 32768$	—	—	1.616	—
Update counts: higher workload 4736 / 1276; lower workload 3008 / 1244.				

Grouping uses a fixed multiset of 32 problems. Offline iteration counts q_t determine the layouts, fixing $\sum_t q_t$ while changing $\sum_w W q_{\max,w}$. Runtime ratios describe the resulting effect for this problem set. The layouts provide a controlled analysis of completion variation.

A.5 Reproducibility

The experiment artifacts record input and source identities, software and GPU environments, commands, timings, and output checks. Per-seed measurements and paired analyses are available at `https://github.com/cis-aconiticacid/DrainSinkhorn`. Figure sources and plotting scripts accompany the manuscript.